

Digital Egypt Pioneers Initiative

Round 4
2025 - 2026

رواد مصر الرقمية

الدورة الرابعة
2025 - 2026

Track: AI & Data Science - Microsoft Machine Learning Engineer

Group Code: ONL4_AIS2_S7

Instructor: Eman Elgalad

Company Name: Skills Dynamix

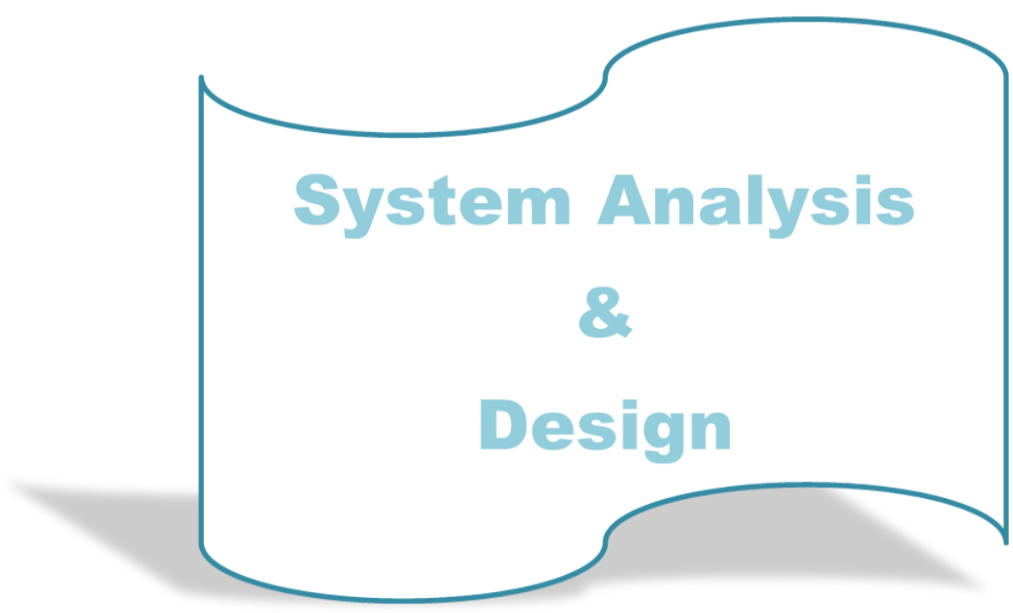


Project Name

"AI-Powered Predictive Maintenance for Industrial Equipment"

No.	Name
1	Moaz Osama Esamil Hataba (TL)
2	Abrar Ashraf Shaban Ahmad
3	Marwan Muhammad Darwish Ammar
4	Yussef Ahmad Muhammad Elshamy
5	Salma Muhammad Nageh Abdul-Hamid
6	Nour-Eldin Kamel Muhammad Hasanein

Task	Page No.	Evaluation
System Actors & Interactions	5	
Use Cases	7	
Use Case Diagram	10	
Functional Requirements	11	
Non-Functional Requirements	13	
System Capabilities	15	
System Constraints	16	
Architecture Style	17	
High-Level System Components	18	
System Interactions	20	
Software Architecture	21	



System Analysis & Design

System Actors & Interactions:

● Maintenance Engineers

- Role:
 - Maintenance engineers are the primary users of the predictive maintenance system. They are responsible for monitoring equipment health, scheduling maintenance activities, and minimizing unexpected failures.
- Responsibilities:
 - Monitor equipment health status.
 - Analyze Remaining Useful Life (RUL) predictions.
 - Schedule maintenance operations.
 - Monitor equipment degradation through simulated real-time monitoring.
- Needs:
 - Accurate Remaining Useful Life predictions.
 - Real-time monitoring dashboard.
 - Explainable prediction results.
 - Maintenance recommendations.
 - Automatic real-time equipment monitoring.
 - Continuous Remaining Useful Life updates.
- Priority:
 - High.

● Plant Managers

- Role:
 - Plant managers oversee operational performance and maintenance planning to ensure efficient production and minimal downtime.
- Responsibilities:
 - Monitor operational performance.
 - Approve maintenance schedules.
 - Evaluate maintenance costs.
 - Improve operational efficiency.
- Needs:
 - Equipment health reports.
 - Downtime statistics.
 - Performance metrics.
 - Maintenance planning support.
 - Operational dashboards for decision support.
- Priority:
 - High.

- **System Administrators**

- Role:
 - System administrators are responsible for deploying, maintaining, and monitoring the predictive maintenance platform.
- Responsibilities:
 - Deploy backend services.
 - Monitor API availability.
 - Ensure system reliability.
 - Monitor application logs.
- Needs:
 - Stable deployment.
 - Secure APIs.
 - Reliable performance.
 - Error monitoring.
- Priority:
 - Medium.

- **Machine Learning Engineers**

- Role:
 - Machine learning engineers develop and improve machine learning models for Remaining Useful Life prediction.
- Responsibilities:
 - Analyze data.
 - Engineer features.
 - Train machine learning models.
 - Evaluate model performance.
 - Implement Explainable AI.
 - Monitor model performance during simulated real-time prediction.
- Needs:
 - High-quality datasets.
 - Feature importance analysis.
 - Performance metrics.
 - Model explainability.
- Priority:
 - High.

Use Cases:

● UC-1: Dataset Selection

- Primary Actor
 - Maintenance Engineer.
- Preconditions
 - The NASA CMAPSS dataset is available.
 - The system is initialized.
- Main Flow
 - User selects FD001–FD004.
 - User selects Engine ID.
 - System validates selection.
 - System validates the selected dataset and engine.
 - Selected data is prepared for simulation and prediction.
- Output
 - Selected dataset.

● UC-2: Automatic Real-Time Monitoring

- Primary Actor
 - Maintenance Engineer.
- Preconditions
 - Dataset selected.
 - A trained prediction model is available.
 - Simulation started.
- Main Flow
 - System automatically streams sensor readings.
 - Dashboard updates every cycle.
 - Backend predicts RUL continuously.
 - Equipment health status is updated.
 - Visualization refreshes automatically.
- Output
 - Continuous equipment monitoring.

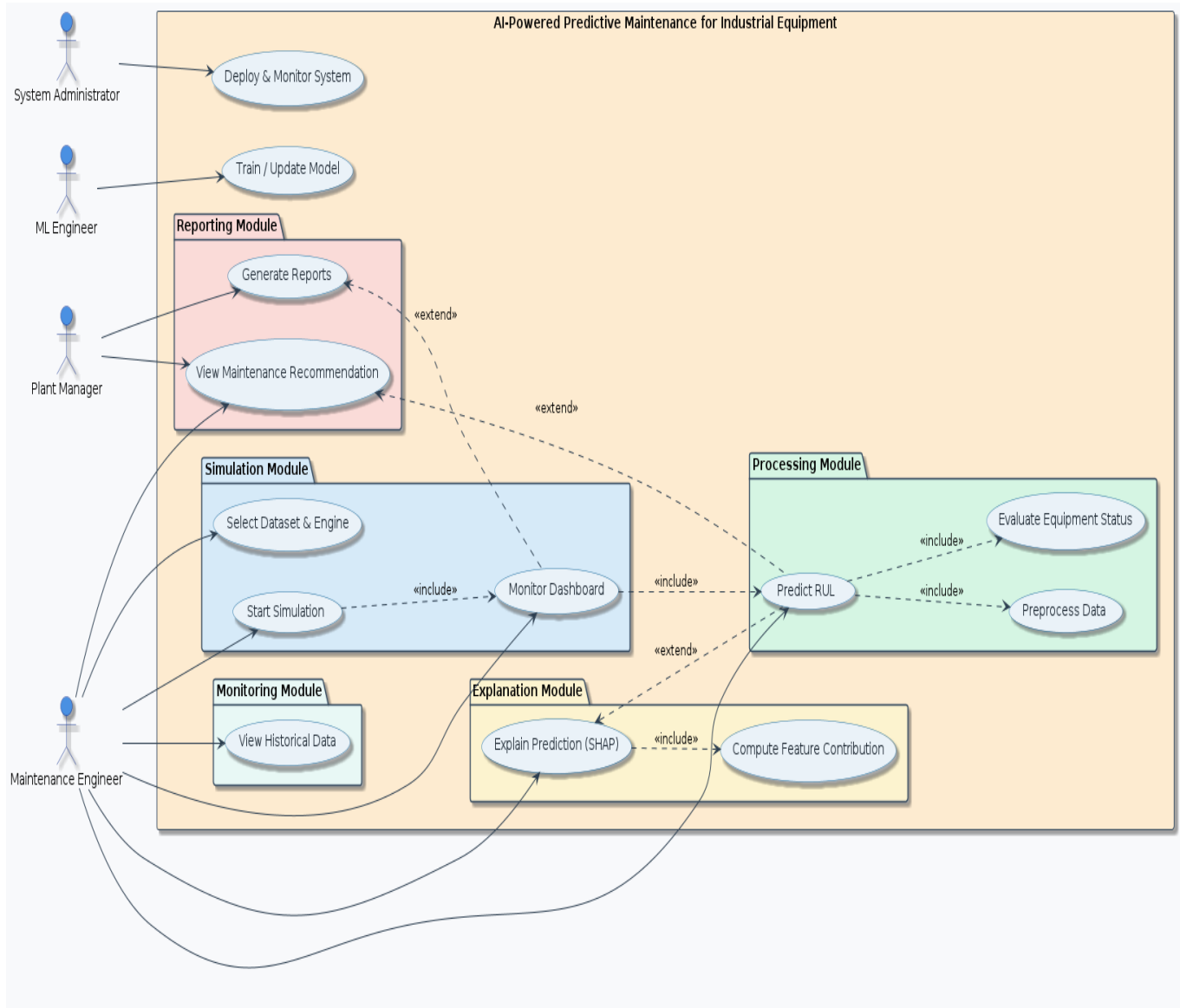
● UC-3: Predict Remaining Useful Life (RUL)

- Primary Actor
 - Maintenance Engineer.
- Preconditions
 - A trained machine learning model is available.
 - A NASA CMAPSS dataset scenario has been selected.
 - The backend API is running and accessible.

- Main Flow
 - The system automatically loads the corresponding sensor data.
 - The backend preprocesses incoming data.
 - The machine learning model predicts the Remaining Useful Life (RUL).
 - The dashboard automatically displays updated predictions.
- Output
 - Predicted Remaining Useful Life (RUL).
- **UC-4: Dashboard Visualization**
 - Primary Actor
 - Maintenance Engineer.
 - Preconditions
 - A prediction has been generated.
 - The dashboard is connected to the backend service.
 - Main Flow
 - Dashboard receives prediction.
 - Dashboard updates charts.
 - Dashboard updates gauges.
 - Dashboard updates health indicator.
 - Dashboard displays SHAP visualization.
 - Output
 - Updated dashboard.
- **UC-5: View Historical Sensor Data**
 - Primary Actor
 - Maintenance Engineer.
 - Preconditions
 - Historical sensor data is available.
 - Main Flow
 - The user selects dataset scenario.
 - The user selects engine.
 - The user selects operational cycle range.
 - The system retrieves historical sensor measurements.
 - The dashboard displays sensor trends and degradation visualizations.
 - Output
 - Historical sensor readings and trend analysis.
- **UC-6: Explain Prediction**
 - Primary Actor
 - Maintenance Engineer.
 - Preconditions

- A prediction has been generated.
- Main Flow
 - The user requests an explanation for the prediction.
 - The SHAP framework computes feature contributions.
 - The dashboard visualizes feature importance and SHAP values.
 - The dashboard highlights the most influential sensors affecting the prediction.
 - The user reviews and interprets the factors influencing the prediction.
- Output
 - Feature importance analysis and SHAP explanations.
- **UC-7: Review Maintenance Recommendation**
 - Primary Actor
 - Maintenance Engineer.
 - Preconditions
 - A Remaining Useful Life (RUL) prediction has been generated.
 - Equipment health status has been evaluated.
 - Main Flow
 - User reviews RUL prediction.
 - User reviews equipment health status.
 - System provides maintenance recommendation.
 - User reviews the recommendation and decides whether maintenance actions should be initiated.
 - Output
 - Maintenance recommendation.
- **UC-8: Generate Reports**
 - Primary Actor
 - Plant Manager.
 - Preconditions
 - Prediction and maintenance data are available.
 - Main Flow
 - The user selects the report type.
 - The system generates the requested report.
 - The report includes prediction summaries, model performance metrics, explainability analysis, and maintenance recommendations.
 - Output
 - Maintenance and performance reports.

Use Case Diagram:



Functional Requirements:

● Dataset & Simulation

- FR-01
 - The system shall allow users to select a NASA CMAPSS dataset scenario (FD001–FD004).
- FR-02
 - The system shall allow users to select an engine from the selected dataset.
- FR-03
 - The system shall automatically load and validate the corresponding sensor data before starting the simulation.
- FR-04
 - The system shall simulate real-time sensor data streaming by sequentially replaying operational cycles from the selected NASA CMAPSS dataset.

● Prediction Engine

- FR-05
 - The system shall preprocess incoming sensor data using the same preprocessing pipeline applied during model training.
- FR-06
 - The system shall predict the Remaining Useful Life (RUL) using the trained machine learning model.
- FR-07
 - The system shall continuously update the Remaining Useful Life (RUL) prediction after each simulated operational cycle.
- FR-08
 - The system shall classify equipment health status (Healthy, Warning, or Critical) based on the predicted Remaining Useful Life (RUL).

● Explainable AI

- FR-09
 - The system shall generate SHAP-based explanations for each prediction to identify the contribution of individual sensor features.
- FR-10
 - The dashboard shall display SHAP values, feature importance rankings, and explanation visualizations for every prediction.

- **Dashboard**

- FR-11
 - The system shall provide an interactive Streamlit dashboard for monitoring equipment health and prediction results.
- FR-12
 - The dashboard shall automatically update sensor readings during simulation.
- FR-13
 - The dashboard shall display (RUL, Equipment Health Status, Sensor Trends, SHAP Explanations, & Current Simulation Cycle).

- **Historical Analysis**

- FR-14
 - The system shall allow users to view historical sensor data.
- FR-15
 - The system shall visualize equipment degradation trends across operational cycles using interactive charts.

- **Backend**

- FR-16
 - The backend shall expose RESTful API endpoints for prediction requests and dashboard communication.
- FR-17
 - The backend shall send prediction results, equipment health status, and SHAP explanations to the dashboard in real time during simulation.
- FR-18
 - The system shall load the trained machine learning model automatically when the backend service starts.

- **Reports**

- FR-19
 - The system shall generate downloadable maintenance and performance reports.
- FR-20
 - The generated reports shall include (Prediction Summary, Model Performance Metrics, SHAP Explainability Analysis, Equipment Health Status, & Maintenance Recommendation).

Non-Functional Requirements:

- **Performance**

- NFR-01
 - The system shall generate Remaining Useful Life (RUL) predictions within one second after receiving valid sensor data under normal operating conditions.
- NFR-02
 - The dashboard shall load completely within three seconds under normal operating conditions.

- **Availability**

- NFR-03
 - The system shall remain available and responsive throughout each simulation session without unexpected interruptions.

- **Reliability**

- NFR-04
 - The system shall produce consistent prediction results when processing identical input data and model configurations.

- **Security**

- NFR-05
 - The system shall validate and sanitize all user inputs, rejecting invalid dataset selections, engine IDs, and malformed requests before processing.

- **Usability**

- NFR-06
 - The system shall provide a simple and intuitive user interface.
- NFR-07
 - Predictions, equipment health status, SHAP explanations, simulation progress, and visualizations shall be clearly displayed and easy to interpret.

- **Maintainability**

- NFR-08
 - The codebase shall follow PEP8 coding standards.
- NFR-09
 - The system shall follow a modular architecture to support maintainability and extensibility.

- **Scalability**

- NFR-10
 - The system architecture shall support adding new datasets and machine learning models with minimal code modifications.

- **Portability**

- NFR-11
 - The system shall be executable on Windows, Linux, and macOS environments supporting Python.

- **Compatibility**

- NFR-12
 - The system shall be compatible with modern web browsers including Google Chrome, Microsoft Edge, and Mozilla Firefox.

- **Explainability**

- NFR-13
 - The system shall provide clear and consistent SHAP-based explanations for every prediction.

- **Documentation**

- NFR-14
 - The source code shall include clear comments, API documentation, and developer documentation to facilitate future maintenance and extension.

- **Reproducibility**

- NFR-15
 - The system shall produce reproducible prediction results when executed using the same trained model, preprocessing pipeline, and input data.

System Capabilities:

- **Data Simulation & Monitoring**

- The system is capable of simulating real-time sensor data by replaying CMAPSS dataset operational cycles.
- The system can continuously monitor equipment conditions during simulation execution.

- **Predictive Analytics**

- The system can estimate Remaining Useful Life (RUL) for equipment using a trained machine learning model.
- The system can classify equipment health status into different operational states (Healthy, Warning, Critical).

- **Data Visualization**

- The system provides interactive visualization of sensor readings and degradation trends.
- The system displays real-time dashboard updates during simulation execution.

- **Explainability**

- The system can generate explanations for predictions using feature contribution analysis (SHAP-based interpretation).
- The system can highlight the most influential sensor features affecting model output.

- **Reporting & Decision Support**

- The system can generate maintenance and performance reports.
- The system supports maintenance decision-making by providing prediction insights and recommendations.

System Constraints:

- **Data Constraints**

- The system is limited to the NASA CMAPSS dataset (FD001–FD004).
- System performance depends on the quality and structure of the provided dataset.

- **Model Constraints**

- Predictions are dependent on a pre-trained machine learning model.
- Model accuracy is limited by training data quality and feature engineering.

- **Architectural Constraints**

- The system relies on a REST-based backend architecture for communication.
- The system uses a modular pipeline (preprocessing → prediction → explanation).

- **Runtime Constraints**

- The system operates under simulated real-time conditions, not actual hardware sensor streams.
- Performance depends on computational resources available on the host machine.

- **Platform Constraints**

- The system is implemented in Python and must run in a Python-supported environment.
- The system is compatible with modern web browsers for dashboard access.

- **Dependency Constraints**

- The system depends on external libraries for:
 - Machine learning (e.g., scikit-learn / XGBoost).
 - Explainability (SHAP).
 - Web framework (FastAPI / Streamlit).

Architecture Style:

- **The system follows a:**
 - Layered Architecture (3-Tier + ML Pipeline Integration) with a modular separation of concerns:
 - Presentation Layer.
 - Application Layer (Backend Services).
 - Machine Learning & Explainability Layer.
 - Data Layer.
- **This hybrid design ensures:**
 - Scalability.
 - Maintainability.
 - Separation of responsibilities.
 - Easy extension for new models or datasets.

High-Level System Components:

- **Presentation Layer**

- Technology:
 - Streamlit Dashboard.
- Components:
 - Display real-time system outputs.
 - Visualize sensor data and RUL predictions.
 - Show SHAP explanations and trends.
 - Allow user interaction (dataset selection, engine selection).

- **Application Layer**

- Technology:
 - FastAPI.
- Components:
 - API Controller
 - Handles all client requests.
 - Manages communication between frontend and backend modules.
 - Simulation Engine
 - Simulates real-time behavior by replaying CMAPSS dataset cycles.
 - Controls timing and streaming of sensor data.
 - Report Generator
 - Produces downloadable reports.
 - Aggregates predictions, metrics, and explanations.

- **Machine Learning & Explainability Layer**

- Components:
 - Preprocessing Pipeline
 - Cleans and transforms raw sensor data.
 - Ensures consistency with training format.
 - RUL Prediction Model
 - Predicts Remaining Useful Life for equipment.
 - Core predictive engine of the system.
 - SHAP Explainer
 - Generates feature importance values.
 - Explains model predictions for interpretability.

- **Data Layer**

- Components:
 - NASA CMAPSS Dataset (FD001–FD004)

- Source of operational sensor data.
- Trained ML Model
 - Stored trained model used for inference.
- Configuration Files
 - System parameters (thresholds, settings, simulation configs).

System Interactions:

- **Step 1: Simulation Start**
 - User selects dataset + engine.
 - Simulation Engine loads CMAPSS data.
- **Step 2: Data Streaming**
 - Sensor data is streamed cycle-by-cycle.
 - Sent to backend processing pipeline.
- **Step 3: Processing Pipeline**
 - Data is preprocessed.
 - Passed to ML model for inference.
- **Step 4: Prediction Stage**
 - RUL is predicted.
 - Equipment health status is determined
- **Step 5: Explainability Stage**
 - SHAP computes feature contributions.
 - Results sent to backend.
- **Step 6: Presentation Layer**
 - Dashboard updates in real time:
 - RUL.
 - Health status.
 - Sensor trends.
 - SHAP explanations.
- **Step 7: Reporting**
 - System generates downloadable reports on demand.

Software Architecture:

Predictive Maintenance Enterprise-Level Architecture Diagram

Based strictly on MVC (Model-View-Controller) pattern

